

FedActuary

Product Requirements Document v1.0 (Implementation-Ready)

August 2026

Status: Supersedes draft v0.1. Consolidates all corrections and locked decisions from the verification, methodology, and abstract-finalization work preceding implementation. This is the single source of truth for Phase 1 onward — where this document and any earlier one disagree, this document is correct.

Contents

1. Overview
2. Problem Statement and Benchmark
3. Objectives
4. Scope
5. Verified Ground Truth
6. System Architecture
7. Locked Parameters
8. Functional Requirements by Phase
9. Evaluation Metrics
10. Tech Stack
11. Team and Roles
12. Sandbox Constraints for Implementing Agents
13. Risks
14. Deliverables Checklist
15. Known Doubts — Phases 2–5
16. Change Log

1. Overview

Ten simulated insurers jointly train a car-insurance claim-frequency model via horizontal federated learning (FL), without sharing raw policyholder data. The project reproduces the baseline of Śmietanka et al. [1] and extends it along three axes the base study leaves untested: realistic (non-IID) data heterogeneity, formal differential privacy, and post-hoc explainability under both.

2. Problem Statement and Benchmark

Insurers cannot legally or competitively pool raw claims data. FL closes most of the gap between full pooling and isolated training. Base-study benchmark (% Poisson Deviance Explained, %PDE — higher is better):

Scenario	%PDE
Global (full pooling)	5.57%
Federated (FedAvg)	5.34%
Partial (no collaboration)	3.20–3.82%

These are the reference values Phase 1 must reproduce the pattern of.

3. Objectives

1. Reproduce Global, Partial, and Federated baseline scenarios; validate the reproduced pattern against Section 2.
2. Quantify the effect of realistic non-IID data distribution on federated performance; evaluate whether FedProx mitigates it relative to FedAvg.
3. Integrate differential privacy (DP-SGD via Opacus); quantify the accuracy–privacy trade-off across a target ϵ range.
4. Evaluate whether SHAP feature-attribution rankings stay consistent across centralized, federated, heterogeneous, and privacy-constrained model variants.

4. Scope

In scope: horizontal FL (FedAvg, FedProx); Dirichlet-based non-IID partitioning; Opacus DP-SGD across a target ϵ sweep; a frozen-split per-layer clipping extension (Section 7); SHAP (GradientExplainer only); %PDE and Gini evaluation; Streamlit dashboard with precomputed default view, a model-details tab, a comparative real-time prediction tool (Section 8.1), and an opt-in reduced-scale live demo; deployment to a free hosting platform (Section 10.1).

Out of scope — explicit, not to be added mid-implementation regardless of perceived low cost: rebuilding the base study’s custom SMPC protocol; homomorphic encryption; vertical FL; real multi-organization deployment; federated gradient boosting; adversarial/poisoning attack simulation; **subgroup or fairness analysis across driver-age or BonusMalus bands** (explicitly evaluated and excluded from core scope — see Change Log); **generic batch/CSV upload scoring** (production-app territory — authentication, rate limiting, error handling at scale — evaluated and excluded as outside this project’s pedagogical scope); any dataset other than freMTPL2freq.

5. Verified Ground Truth

Confirmed directly against the base study’s reference implementation (github.com/actuari/IFoA-FL-WP), not its text alone. Any implementation disagreeing with this section is wrong and should be corrected to match, not the reverse.

Dataset: freMTPL2freq.csv — 678,013 rows, 12 raw columns, 0 missing values, 22 Region categories. Raw ClaimNb maximum is 16 (not 11).

Preprocessing, exact order:

1. ClaimNb $\leftarrow$ clip(upper=4)

2. VehAge $\rightarrow$ clip(upper=20)
3. DrivAge $\rightarrow$ clip(upper=90)
4. BonusMalus $\rightarrow$ clip(upper=150)
5. Density $\rightarrow$ log-transform, **before** scaling
6. Exposure $\rightarrow$ clip(upper=1)
7. Drop IDpol
8. Area $\rightarrow$ map A-F to 1-6
9. VehGas $\rightarrow$ map {Regular: 1, Diesel: 2} $\rightarrow$ not {0, 1}
10. One-hot encode VehBrand, Region, drop_first=True
11. Three-way split $\rightarrow$ train $\approx$ 72% / val $\approx$ 8% / test $\approx$ 20% $\rightarrow$ via `train_test_split(test_size=0.2)` then `train_test_split(test_size=0.1)` on the remainder. **Verified against the base study's own `split_data()` function** $\rightarrow$ matches exactly, confirmed by running against the real 678,013-row file (train=488,169, val=54,241, test=135,603)
12. MinMax-scale [Area, VehPower, VehAge, DrivAge, BonusMalus, Density] to [0, 1], **fit on the training split only, applied to val and test** $\rightarrow$ avoids leaking held-out statistics into the scaler; the base study's own code scales before splitting instead, a small real leakage risk this implementation deliberately does not replicate (PRD Section 10 principle: reproduce the pattern, not bit-identical output)
13. Output: X $\rightarrow$ **39 features, Exposure included as a feature, not held out as a Poisson-regression offset**; y = ClaimNb; exposure also returned separately, required for %PDE weighting (Section 9); fitted scaler and feature-column order persisted per Section 8/12.10

A preprocessing implementation producing a feature count other than 39 has a bug and must be fixed before proceeding $\rightarrow$ do not adjust the target count to match a broken implementation.

Model architecture — MultipleRegression:

Input(39) $\rightarrow$ Linear(39,15) $\rightarrow$ Tanh $\rightarrow$ Linear(15,5) $\rightarrow$ Tanh $\rightarrow$ Linear(5,1) $\rightarrow$ exp()

Xavier-uniform weight initialization, zero bias initialization. Optimizer: NAdam. Loss: `PoissonNLLLoss(log_input=False, full=True)`, target y = ClaimNb (raw count) $\rightarrow$ the model's output is compared **directly** against the count, not multiplied by exposure; Exposure earns its effect only as one of the 39 input features. Verified directly against `insur_FL_client.py/insur_FL_server.py`, the scripts that actually produced the paper's published numbers $\rightarrow$ **not** `architecture.py`'s class defaults, which are (25, 2) and are never actually called with those defaults anywhere in the real experiment code. (15, 5) is the output of the paper's own hyperparameter search (`skorch_tuning.py`, swept {5, 10, 15, 20} for both units). This is a correction, not a first pass $\rightarrow$ see Change Log.

6. System Architecture

Six sequential phases. Phase 1 establishes the baseline. Phases 2 and 3 branch independently off the Phase 1 federated baseline and are never evaluated in combination with each other's parameter sweep. Phase 4 consumes saved checkpoints from Phases 1, 2, and 3. Phases 5 and 6 conclude the pipeline.

Eight architecture diagrams (overview; preprocessing pipeline; model architecture; FedAvg/FedProx mechanism; Dirichlet partitioning mechanism; DP-SGD mechanism with per-layer extension; SHAP workflow; dashboard architecture) were prepared separately and are not reproduced here — see project diagram set.

7. Locked Parameters

Parameter	Value	Notes
Dirichlet α — ClaimNb split	{0.1, 0.5, 1, 5}	Full sweep, extreme to near-uniform
Dirichlet α — Region split	0.5	Single value, not swept
Privacy targets (ϵ)	$\{\infty, 8, 3, 1\}$	$\delta = 1 \times 10^{-5}$ throughout. Targets tuned toward via noise multiplier + schedule, then measured via accountant — not set directly
Per-layer DP clipping	Frozen split, computed once from Phase 1 checkpoint	Extension cell within Phase 3, not core sweep. Same ϵ as flat clipping by construction — see Methodology §8.4
FedProx proximal term	Client-side only	Server-side strategy swap alone does not implement FedProx
Baseline training schedule	350 rounds $\times$ 10 local epochs	Phase 1 only — corrected from 300, verified against <code>run_config.py</code> 's actual <code>server_config</code> . Phase 3 uses an independently tuned, shorter schedule
Batch size	500	Verified against <code>run_config.py</code> . Mini-batch via <code>DataLoader(shuffle=True)</code> , not full-batch — confirmed from <code>insur_FL_client.py</code>
Learning rate	0.001	Verified against <code>run_config.py</code> (LEARNING_RATE)
SHAP background sample size	≤ 50	Newly verified — see Section 8 Phase 4 and Change Log

8. Functional Requirements by Phase

Phase	Requirement	Output
0	Environment setup; data acquisition; 2-client/3-round smoke test	Done. Verified working; loss trended 0.68–0.47–0.37
1	Global / Partial / Federated per Section 5–6; persist every model + a per-round metrics CSV + the fitted preprocessing scaler and ordered feature-column list. Equal effective training budget across all three, not equal epoch count — Federated clients accumulate rounds $\times$ local_epochs of local training on their own data over the full run; Global and Partial must be given a matching budget (rounds epochs on pooled data; rounds $\times$ local_epochs epochs on each insurer's own shard respectively), or the comparison is structurally unfair. See Change Log	Baseline %PDE results
2	Both Dirichlet splits (Section 7); FedAvg vs. correctly client-side FedProx	Heterogeneity-vs-%PDE curve
3	Opacus DP-SGD across ϵ targets; independently tuned schedule; optional per-layer cell	Privacy-vs-%PDE curve
4	GradientExplainer only — DeepExplainer confirmed broken (fails at every background size tested: 1, 10, 50, 100, all with the same tensor-shape error). GradientExplainer confirmed working, but only at background sample size ≤ 50 — it fails the same way at 100. Keep background samples ≤ 50 as a safe margin	Feature-ranking comparison across 5 checkpoints

Phase	Requirement	Output
	(exact threshold between 51–99 not pinned down, not worth the extra verification time). Applied on Global, Federated-baseline, one non-IID checkpoint, two DP checkpoints	
5	Streamlit app reading results/*.csv + checkpoints/*.pt; default view fully offline; model-details tab; comparative real-time prediction tool (Section 8.1); opt-in live-demo button at reduced scale (3 insurers, 10–15 rounds, 2 local epochs); deployed per Section 10.1	Working, deployed dashboard
6	Repo with one-command reproduction; report structured motivation—method—experiments—results—limitations; demo video + backup recording	Submission package

8.1 Phase 5 Detail — Dashboard Extensions

- **Model-details tab.** Architecture summary, training configuration, and resulting %PDE/Gini per scenario (Global, Partial, Federated, each DP ϵ level). Reads existing results//checkpoints/ artifacts only — no new infrastructure required.
- **Comparative real-time prediction tool.** A form for a single policy’s 12 raw fields. On submission, runs the shared preprocessing function once and produces side-by-side predictions from the Global, Federated, and one DP- ϵ checkpoint, each paired with a SHAP attribution breakdown via the Phase 4 GradientExplainer pipeline (reused, not reimplemented; background sample ≤ 50 per Section 7). This is the project’s actual research question made interactive, not a generic model-serving demo.
- **Persistence requirement, hard constraint.** The fitted MinMaxScaler and the ordered list of 39 training-time feature columns must be loaded from the Phase 1 artifacts specified in Section 8’s Phase 1 row, never refit on the single incoming record — a scaler fit on one row produces meaningless 0/1 values for every column. See also Section 12, item 10.
- **Explicitly excluded:** batch/CSV upload scoring (Section 4).

- **Stretch, not core, no timeline impact if skipped:** a continuous ϵ slider in place of fixed checkpoint selection; a Dirichlet- α heterogeneity explorer visualizing one insurer's data distribution at a chosen skew.

9. Evaluation Metrics

Primary: % Poisson Deviance Explained (%PDE), computed via scikit-learn's `d2_tweedie_score(y_true, y_pred, sample_weight=exposure, power=1) × 100` — verified directly against the base study's own evaluation code (`skorch_tuning.py`), which calls this exact function this exact way, rather than a hand-derived formula. `y_pred` is the model's raw output, compared directly against `y_true = ClaimNb`; `exposure` enters only as `sample_weight`, not as a multiplier on the prediction (Section 5). Higher is better. Implemented in `src/metrics.py` and sanity-tested: a flat constant prediction (matching `d2_tweedie_score`'s own internal null) scores $\approx 0\%$ by construction; a rate-oracle prediction clears the real published Global benchmark of 5.57%.

Secondary: Gini coefficient — ranking ability, computed by ordering policies on **implied frequency** (`y_pred / exposure`), not raw predicted count — ranking by raw count would systematically favor long-exposure policies regardless of actual risk. Tracked because average deviance can stay stable while ranking ability degrades, given $\sim 95\%$ of policies record zero claims. Not subgroup-sliced (Section 4).

10. Tech Stack

Component	Library	Version	Note
Language	Python	3.10+	
DL framework	PyTorch	2.14.0	Verified installed (Phase 1 implementation pass)
FL orchestration	Flower (<code>flwr[simulation]</code>)	1.37.0	Verified installed; requires ray for simulation mode
Non-IID partitioning	<code>flwr-datasets</code>	0.5.0	Verified installed
Differential privacy	Opacus	1.6.0	
Explainability	SHAP	0.52.0	GradientExplainer only, background ≤ 50 — Section 7
Dashboard	Streamlit	latest stable	
Data handling	pandas, NumPy, scikit-learn	latest stable	

Current versions used throughout — **not** the base study's pinned versions (`flwr==1.1.0`, `torch==1.13`, legacy `fl.server.start_server` API). “Current” drifts as upstream releases ship; the version actually used is whatever `requirements.txt` pins at implementation time, re-verified per Section 12 item 2 — the numbers above reflect the most recent verification pass,

not a permanent pin. Known harmless conflict: flwr requires rich>=14, flwr-datasets requires rich<14 — cosmetic, confirmed non-blocking.

10.1 Deployment Platform

Platform	Free tier	Verdict
Streamlit Community Cloud	1 vCPU, ~1 GB RAM; sleeps on inactivity, wakes on visit	Chosen. Purpose-built for this exact stack; connects directly to the GitHub repo, zero config beyond requirements.txt
Hugging Face Spaces	2 vCPU, up to 16 GB RAM (CPU Basic)	Backup only. Community reports (July 2026) of new accounts unable to select free CPU Basic, offered ZeroGPU only — irrelevant here since nothing requires a GPU. Confirm CPU Basic is actually selectable at signup before relying on it
Render	0.1 CPU, 512 MB RAM; sleeps after 15 min, ~1 min cold start	Viable fallback; tightest CPU of the three
Vercel, Netlify	—	Ruled out. Both platforms execute short-lived serverless functions; Streamlit requires a persistent process holding a WebSocket connection for the session lifetime. Architectural mismatch, not a configuration problem

One vCPU is sufficient: Phase 0 measured 10 local epochs on a ~60K-row shard at under 2 seconds, so the live-demo button’s reduced-scale run completes in well under a minute on the chosen tier. The repo’s checkpoints/ and results/ must be committed alongside the app — the dashboard reads them directly and nothing is generated at deploy time.

11. Team and Roles

Owner	Responsibility
Person A	Phase 1 (baseline) → Phase 2 (non-IID + FedProx)
Person B	Phase 3 (differential privacy) — protected schedule buffer; least predictable phase
Person C	Phase 4 (SHAP, startable once Phase 1 checkpoints exist) → Phase 5 (dashboard)
Whole team	Phase 0 (done), Phase 6

Not yet confirmed by the team — asked twice, no answer on record: whether this role split stands as proposed or was divided differently; department-specific submission/methodology template requirements.

12. Sandbox Constraints for Implementing Agents

Binding instructions for any agent (human-directed AI, autonomous coding agent, or otherwise) implementing Phase 1 onward.

1. **Compute.** CPU-only; no GPU required or assumed. Full 300-round $\times$ 10-insurer run $\approx$ 1–2 hours, single-threaded. Use a reduced round/epoch config for all development iteration; reserve the full schedule for final result generation.
2. **Verify before implementing.** Any preprocessing, architecture, or aggregation logic must be checked against the base study’s actual reference implementation (github.com/actuari/IFoA-FL-WP) before being treated as correct. The paper’s prose is not sufficient evidence on its own — Section 5 of this document is the reconciled result of that check and takes precedence over both the paper’s text and the reference repo where they disagree with each other. This extends to claims inherited from earlier planning sessions, not just the paper itself — see Section 15, SHAP re-verification.
3. **Persist everything.** Every phase writes checkpoints to `checkpoints/` and metrics to `results/` as it runs. Nothing may be print-only or held only in memory — Phases 4 and 5 depend on loading saved artifacts rather than retraining.
4. **Config-driven, one codebase.** Insurer count, round count, local epochs, split strategy, and DP on/off are parameters of a single training entry point, not separate scripts. The full run, fast dev iteration, and the live demo are three configs of the same code path.
5. **Locked parameters are not silently adjustable.** Deviating from Section 7 during a pilot run must be flagged explicitly in commit messages or notes, not changed quietly.
6. **No scope additions.** The Section 4 out-of-scope list is binding regardless of implementation-time convenience. “This would be cheap to add” is not sufficient justification — it is the exact reasoning that produced the scope-creep pattern this project was explicitly right-sized against during planning.
7. **Network access.** Do not assume arbitrary URLs are reachable. The reference repo above is confirmed cloneable and contains a working copy of the dataset; prefer it over ad hoc downloads.
8. **Numerical tripwire.** Preprocessing must yield exactly 39 features. A different count indicates a bug in the implementation, not a change in the target.
9. **Library versions.** Use Section 10’s current versions. Do not silently downgrade to the base study’s pinned versions to match its exact numbers — reproduce the *pattern*, not bit-identical output.
10. **Preprocessing artifacts are not optional.** Phase 1 must persist the fitted `MinMaxScaler` and the ordered 39-column feature list, not just the trained model. Phase 5’s comparative prediction tool depends on loading these, not refitting them —

scaling a single new record in isolation against its own min/max is a silent correctness bug, not a missing convenience.

11. **SHAP background sample size.** Keep `GradientExplainer` background samples ≤ 50 . Verified directly: works at 1/10/50, fails with the same tensor-shape error as `DeepExplainer` at 100. Exact threshold between 51–99 not pinned down — not required for implementation, since staying ≤ 50 is a free, safe margin.
12. **Sandbox persistence.** This implementation environment does not reliably persist files or installed packages between sessions. Treat any in-progress code here as ephemeral; the versioned source (this document plus the team’s actual git repo) is the durable record, not this sandbox’s filesystem.

13. Risks

Risk	Mitigation
DP privacy budget compounds fast across rounds $\times$ epochs	Phase 3 uses its own independently tuned schedule (Section 7)
FedProx implemented as server-only strategy swap (common mistake)	Client-side proximal term is a stated requirement (Section 7), verified against Flower’s own source
<code>DeepExplainer</code> incompatible with model architecture	Use <code>GradientExplainer</code> exclusively, background ≤ 50 (both confirmed directly, Section 7/12.11)
Non-IID $\times$ privacy experiment count explodes	Phases 2 and 3 remain independent branches, never crossed (Section 6)
Live demo fails during presentation	Dashboard defaults to precomputed view; live run is opt-in; keep a backup recording
Scope creep re-admitting subgroup/fairness analysis, GBM comparison, etc.	Section 4 and Section 12 item 6 are binding; any addition requires an explicit Change Log entry, not a quiet inclusion
Claims inherited from pre-implementation planning sessions treated as verified without a direct re-check	Section 12 item 2 now applies explicitly to inherited claims, not just the base paper; SHAP re-verification (Section 15) is the first instance of this being caught in practice
Verifying a function’s default parameters without checking whether the real experiment code actually calls it with those defaults	Caught once already (architecture, Section 15) — checking a definition is not the same as checking the call site. Section 12 item 2 extended to require tracing to the actual entry-point script, not just the function/class definition

14. Deliverables Checklist

- ☒ Phase 0 — environment, data, toy run (verified)
- ☐ Public repo — README, architecture diagrams, pinned `requirements.txt`, one-command reproduction

- ☐ results/ — metrics CSVs, Phases 1–4
- ☐ checkpoints/ — all saved models, baseline + non-IID + DP variants
- ☐ Streamlit dashboard — static + model-details tab + comparative prediction tool + live-demo mode
- ☐ Dashboard deployed — Streamlit Community Cloud, live URL
- ☐ Report — motivation, method, experiments, results, limitations
- ☐ Literature review, citations [1]–[5]
- ☐ Abstract — finalized, consistent with this document
- ☐ Demo video + backup recording

15. Known Doubts — Phases 2–5 (Scaffolded, Not Yet Run)

Phases 2–6 were built as a full structural pass after Phase 1 — code written against this document’s locked parameters, not independently re-verified line by line the way Phase 1 was. Listed here explicitly rather than silently assumed correct, per Section 12 item 2’s own standard:

1. **Phase 2 — index alignment bug, likely real, not fixed.** `phase2_nonIID.py` derives a Dirichlet label array by slicing `claimnb_label[:len(X_train)]`, but `split_and_scale` builds `X_train` via `train_test_split`, which reorders rows. The slice almost certainly does not correspond row-for-row to `X_train`’s actual contents. Needs a fix (carry the label through the split by index, not by position) before Phase 2 results can be trusted.
2. **Phase 3 — Opacus integration un-run in this sandbox.** Written from a previously verified API (`PrivacyEngine.make_private`, confirmed in an earlier working session against this exact model), but not executed in this pass — Opacus isn’t installed here. The crude grid-search noise-tuning function (`tune_noise_for_target_epsilon`) is a first pass, not validated against real convergence behavior at the locked round/epoch schedule.
3. **Phase 4 — depends on artifacts Phases 2/3 haven’t actually produced.** Will fail loudly (not silently) on missing checkpoints rather than fabricate a result, but has not been run end-to-end even once.
4. **Phase 5 — prediction tool is a UI skeleton, not wired to inference.** Building a full 39-feature vector from a short form needs defaults for every field the form doesn’t collect (one-hot brand/region, Area, VehGas, Density) — not implemented. Model-details tab reads real Phase 1 output correctly where that output exists.
5. **Partial’s full-scale behavior is genuinely unknown, not just unverified.** Dev-scale evidence (Section 15 Change Log) shows Partial trailing further than the benchmark suggests, plausibly from overfitting on ~3,600 rows/insurer. Whether real per-insurer scale (~48K rows) closes that gap is an open empirical question, not something reasoned through to a confident answer.

None of the above blocks Phase 1 from being considered genuinely validated at dev scale — it blocks Phases 2–5 from being considered anything more than structurally complete.

16. Change Log

Consolidates all corrections made between draft v0.1 and this version:

- Feature count corrected to 39 (was inconsistently 38/39) — Exposure confirmed as an included feature, not a held-out offset.
- Density log-transform added to preprocessing (previously omitted).
- VehGas encoding corrected to {1, 2} (was {0, 1}).
- Raw ClaimNb maximum corrected to 16 (was stated as 11).
- Dirichlet α and ϵ targets finalized (Section 7), removing “to be locked after pilot runs” status from v0.1.
- Per-layer DP redesigned from per-round recomputation (privacy-accounting risk, identified during review) to a frozen split computed once from the Phase 1 checkpoint.
- SHAP scope expanded from 2 checkpoint types (Global, Federated) to include a non-IID checkpoint, matching the project’s own “heterogeneity-robust and explainable” framing — Objective 4, Phase 4, and Section 6 updated accordingly.
- Subgroup/fairness performance analysis evaluated for inclusion and explicitly excluded from core scope (Section 4) — removed from the abstract accordingly.
- Phase 5 dashboard scope expanded: added a model-details tab and a comparative real-time prediction tool (Global/Federated/DP- ϵ side by side with SHAP attribution, Section 8.1). Generic batch/CSV upload scoring evaluated and explicitly excluded (Section 4). Phase 1’s persistence requirement (Section 8, Section 12 item 10) expanded to include the fitted scaler and feature-column list, not just the model, to support this tool.
- Deployment platform selected: Streamlit Community Cloud (Section 10.1), free tier, chosen over Hugging Face Spaces (free CPU-tier availability inconsistently reported as of July 2026) and Render (0.1 CPU free tier — tightest of the viable options). Vercel and Netlify ruled out as architecturally incompatible with Streamlit’s persistent-process requirement.
- Train/test protocol added during Phase 1 implementation (Section 5, steps 11–12): initially assumed as an unverified 80/20 default, then checked directly against the base study’s `split_data()` function and corrected to the real protocol — a two-stage split giving train $\approx$ 72%/val $\approx$ 8%/test $\approx$ 20%, confirmed by running it against the actual 678,013-row file. Scaler still fit on training rows only (deliberate deviation from the base study, which scales before splitting).
- Tech stack versions (Section 10) updated to the versions actually verified during the Phase 1 implementation pass (PyTorch 2.14.0, Flower 1.37.0, up from 2.13.0/1.33.0) — expected drift from “latest stable,” not a substantive change.

- SHAP explainer claim re-verified rather than left as an inherited, unchecked fact from the pre-implementation planning session. Result: more precise than the original claim, not simply confirmed. DeepExplainer fails at every background size tested (1, 10, 50, 100) — the original “broken” claim holds. GradientExplainer works at 1/10/50 but fails the same way at 100 — the original “works” claim was correct but incomplete; it didn’t document a background-size ceiling because that constraint was never actually tested until now. New locked constraint: keep background samples ≤ 50 (Section 7, Section 12 item 11).
- **Architecture corrected: (15, 5) hidden units, not (25, 2).** The earlier verification checked `architecture.py`’s class definition and stopped there — those are generic defaults, never actually called with those values anywhere in the real experiment code. `insur_FL_client.py` and `insur_FL_server.py` — the scripts that produced the paper’s published benchmark numbers — explicitly instantiate the model with `run_config.py`’s tuned values, (15, 5), the result of the paper’s own hyperparameter search. `src/model.py` and PRD Section 5 corrected accordingly.
- **Training schedule corrected: 350 rounds, not 300** — verified against `run_config.py`’s `server_config`, previously assumed rather than checked.
- **Batch size (500) and learning rate (0.001) added to Section 7** — verified against `run_config.py`; previously absent from this document entirely. Training is mini-batch (`DataLoader`, shuffled), not full-batch, confirmed from `insur_FL_client.py`.
- **Training-budget fairness bug found and fixed by actually running Phase 1, not by review.** First dev-scale run gave Global a flat epochs (10) training budget — identical to one Federated client’s *per-round* local budget — while Federated accumulated rounds $\times$ epochs of cumulative local training across the full run. Result: Global scored below Federated (structurally impossible — Global sees strictly more data with no averaging penalty). Fixed: Global now trains for rounds epochs on the pooled set; Partial insurers now train for rounds $\times$ epochs on their own shard, matching the cumulative local exposure a Federated client gets. After the fix, a dev-scale run (50K rows, 50 rounds) produced the first run with the *correct relative ordering* — Global (2.61%) > Federated (2.06%) > Partial (avg -4.57%) — matching the paper’s own Global > Federated > Partial relationship, though absolute magnitudes are still below the full benchmark (5.57/5.34/3.2–3.82), expected at $\sim 7\%$ of the real data volume. Partial still trails further than the benchmark suggests; plausibly real overfitting (500 epochs on 3,600 rows/insurer) that full-scale per-insurer data (~ 48 K rows) should ease, not yet confirmed.
- **Evaluation metric corrected: model output is a predicted count, compared directly to $y = \text{ClaimNb}$, not a frequency requiring multiplication by exposure.** Verified against `split_data()` (target is literally `df_encoded.iloc[:, 0]`, i.e. `ClaimNb`) and the training loop’s `PoissonNLLLoss(log_input=False, full=True)` call, which compares prediction to that count directly. `src/metrics.py` rewritten to use `scikit-learn`’s `d2_tweedie_score(power=1)` with exposure as `sample_weight`, matching `skorch_tuning.py`’s own evaluation call exactly, replacing an earlier hand-derived formula built on the wrong mental model. Gini’s

ranking variable corrected to implied frequency ($y_{\text{pred}} / \text{exposure}$) for the same reason. PRD Section 9 rewritten accordingly.

- Phases 2–6 scaffolded in a single fast pass (partitioning, FedProx, Opacus DP-SGD, SHAP, dashboard skeleton, README/requirements) — breadth over per-phase verification depth, by deliberate choice. Every phase beyond Phase 1 carries explicit, named doubts rather than being presented as equally solid — see Section 15.

References

[1] M. Śmietanka, D. Liew, S. Hand, H. H. Loh, and Y.-Y. M. Chen, “Privacy preserving neural network predictive modelling in insurance using horizontal federated learning,” *British Actuarial Journal*, vol. 31, e8, pp. 1–37, 2026, doi: 10.1017/S1357321725100329.